IFCT0109 · MF0489_3

CRIPTOGRAFÍA

Unidad Didáctica 1 · Cuaderno de Prácticas

OpenSSL · GPG · Hashing · Firma Digital en Kali Linux

Alumno/a: _____	**Fecha:** _____
Grupo: _____	**Calificación:** _____

INFO Este cuaderno contiene 5 ejercicios prácticos realizados sobre Linux. Lee el enunciado completo antes de ejecutar ningún comando.

Objetivos de la práctica

- Aplicar algoritmos de cifrado simétrico y asimétrico con OpenSSL
- Generar y verificar hashes criptográficos con SHA-256 y SHA-512
- Crear y gestionar claves GPG para cifrado y firma de mensajes
- Comprender en la práctica el efecto avalancha de las funciones hash
- Firmar digitalmente archivos y verificar la autenticidad de la firma

Entorno de trabajo

Todos los ejercicios se realizan en la máquina Kali Linux accesible desde Guacamole. Abre una terminal y verifica que tienes las herramientas disponibles:

```
# Verificar versión de OpenSSL
openssl version

# Verificar GPG
gpg --version | head -3

# Crear directorio de trabajo
mkdir -p ~/practica-cripto && cd ~/practica-cripto
```

Ejercicio 1 Cifrado simétrico con AES-256

20 min
Nivel: Básico

1.1 Contexto

AES (Advanced Encryption Standard) con clave de 256 bits es el estándar de cifrado simétrico más utilizado en la actualidad. En este ejercicio cifrarás y descifrarás un archivo de texto usando AES-256-CBC.

1.2 Preparación

Crea un archivo de texto con contenido de prueba:

```
cd ~/practica-cripto

# Crear archivo con mensaje confidencial
echo 'Este es un mensaje confidencial de prueba.' > mensaje.txt
echo 'Contiene información sensible que debe protegerse.' >> mensaje.txt

# Verificar el contenido
cat mensaje.txt
```

1.3 Cifrado AES-256-CBC

Cifra el archivo usando AES-256 en modo CBC:

```
# Cifrar el archivo (se pedirá contraseña dos veces)
openssl enc -aes-256-cbc -salt -pbkdf2 -in mensaje.txt -out mensaje.enc

# Verificar que el archivo cifrado existe
ls -lh mensaje.txt mensaje.enc

# Intentar leer el archivo cifrado (debería ser ilegible)
cat mensaje.enc
```

NOTA El parámetro -pbkdf2 usa una función de derivación de clave más segura. El parámetro -salt añade aleatoriedad para que el mismo mensaje con la misma clave produzca cifrados diferentes.

1.4 Descifrado

```
# Descifrar el archivo
openssl enc -aes-256-cbc -d -pbkdf2 -in mensaje.enc -out
mensaje_descifrado.txt

# Verificar que el contenido es idéntico al original
diff mensaje.txt mensaje_descifrado.txt && echo 'IDENTICOS' || echo
'DIFERENTES'
cat mensaje_descifrado.txt
```

1.5 Preguntas de reflexión

1. ¿Qué ocurre si introduces una contraseña incorrecta al descifrar? Pruébalo y describe el resultado:

Tu respuesta:

2. ¿Por qué el archivo cifrado tiene un tamaño diferente al original? Busca la razón relacionada con el modo CBC y el padding:

Tu respuesta:

3. ¿Qué problema principal tiene la criptografía simétrica que no tiene la asimétrica?

Tu respuesta:

Ejercicio 2 Funciones Hash y efecto avalancha

20 min
Nivel: Básico

2.1 Contexto

Las funciones hash criptográficas producen una huella digital de longitud fija a partir de cualquier entrada. Una propiedad fundamental es el efecto avalancha: un cambio mínimo en la entrada produce un hash completamente diferente.

2.2 Generación de hashes

```
cd ~/practica-cripto

# Hash SHA-256 del archivo original
sha256sum mensaje.txt

# Hash SHA-512
sha512sum mensaje.txt

# Hash MD5 (obsoleto, solo para comparar)
md5sum mensaje.txt

# Hash con OpenSSL
openssl dgst -sha256 mensaje.txt
openssl dgst -sha512 mensaje.txt
```

2.3 Demostración del efecto avalancha

Vamos a modificar un solo carácter del mensaje y observar cómo cambia completamente el hash:

```
# Guardar el hash original
sha256sum mensaje.txt > hash_original.txt
cat hash_original.txt

# Crear una copia con UN solo carácter diferente
sed 's/confidencial/Confidencial/' mensaje.txt > mensaje_mod.txt

# Comparar los dos archivos
diff mensaje.txt mensaje_mod.txt

# Calcular hash del archivo modificado
sha256sum mensaje_mod.txt

# Comparar ambos hashes
echo '--- Hash original ---'
sha256sum mensaje.txt
echo '--- Hash modificado ---'
sha256sum mensaje_mod.txt
```

2.4 Verificación de integridad

Los hashes se usan para verificar que un archivo no ha sido alterado:

```
# Generar hash de verificación
```

```
sha256sum mensaje.txt > checksums.sha256
cat checksums.sha256

# Verificar integridad (sin alterar el archivo)
sha256sum -c checksums.sha256

# Simular alteración del archivo
echo 'Línea añadida por atacante' >> mensaje.txt

# Verificar de nuevo (debería fallar)
sha256sum -c checksums.sha256
```

2.5 Preguntas de reflexión

1. Anota los hashes SHA-256 del archivo original y el modificado. ¿Qué porcentaje de caracteres cambiaron?

Hash original (SHA-256): Hash modificado (SHA-256):

2. ¿Por qué MD5 ya no se considera seguro para uso criptográfico? ¿En qué situaciones podría seguir usándose?

Tu respuesta:

3. ¿Qué aplicaciones prácticas tiene la verificación de integridad mediante hashes en un entorno empresarial?

Tu respuesta:

Ejercicio 3 Criptografía asimétrica con RSA

25 min
Nivel: Intermedio

3.1 Contexto

RSA es el algoritmo asimétrico más utilizado. Se basa en la dificultad de factorizar números primos muy grandes. En este ejercicio generarás un par de claves RSA y usarás la clave pública para cifrar y la privada para descifrar.

3.2 Generación del par de claves RSA

```
cd ~/practica-cripto

# Generar clave privada RSA de 2048 bits
openssl genrsa -out clave_privada.pem 2048

# Ver la estructura de la clave privada
openssl rsa -in clave_privada.pem -text -noout | head -20

# Extraer la clave pública
openssl rsa -in clave_privada.pem -pubout -out clave_publica.pem

# Ver la clave pública
cat clave_publica.pem

# Ver tamaño de los archivos
ls -lh clave_privada.pem clave_publica.pem
```

⚠ IMPORTANTE

La clave privada NUNCA debe compartirse. La clave pública puede distribuirse libremente. Quien tenga tu clave pública puede enviarte mensajes cifrados que SOLO tú puedes descifrar con tu clave privada.

3.3 Cifrado y descifrado RSA

```
# Crear mensaje pequeño para cifrar con RSA
echo 'Clave AES secreta: X7k9P2mQ' > secreto.txt

# Cifrar con la clave PÚBLICA (cualquiera puede cifrar)
openssl rsautl -encrypt -pubin -inkey clave_publica.pem \
  -in secreto.txt -out secreto.enc

# Verificar que está cifrado
cat secreto.enc

# Descifrar con la clave PRIVADA (solo tú puedes descifrar)
openssl rsautl -decrypt -inkey clave_privada.pem \
  -in secreto.enc -out secreto_descifrado.txt

cat secreto_descifrado.txt
```

3.4 Preguntas de reflexión

1. ¿Por qué RSA solo puede cifrar mensajes pequeños directamente? ¿Cómo se soluciona esto en la práctica (piensa en TLS/HTTPS)?

Tu respuesta:

2. ¿Qué diferencia hay entre el uso de la clave pública y la privada en el cifrado vs. en la firma digital?

Tu respuesta:

Ejercicio 4 Cifrado y firma con GPG

25 min
Nivel: Intermedio

4.1 Contexto

GPG (GNU Privacy Guard) implementa el estándar OpenPGP y combina criptografía simétrica y asimétrica. Es la herramienta estándar para cifrar correos electrónicos, archivos y verificar la autenticidad de software.

4.2 Generación de claves GPG

```
# Generar par de claves GPG
gpg --gen-key
# Introduce: nombre, email y contraseña cuando se solicite
# Nombre sugerido: Alumno Lab   Email: alumno@lab.local

# Ver las claves generadas
gpg --list-keys
gpg --list-secret-keys
```

NOTA GPG usará tu nombre y email como identificador de la clave. En un entorno real, la clave pública se publica en servidores de claves para que otros puedan enviarte mensajes cifrados.

4.3 Cifrado de archivo con GPG

```
cd ~/practica-cripto

# Crear archivo a cifrar
echo 'Informe confidencial - Solo para destinatario autorizado' > informe.txt

# Cifrar con tu propia clave pública (para ti mismo en este ejercicio)
gpg --encrypt --recipient alumno@lab.local --output informe.gpg informe.txt

# Verificar que está cifrado
file informe.gpg
ls -lh informe.txt informe.gpg

# Descifrar
gpg --decrypt informe.gpg
```

4.4 Firma digital con GPG

```
# Firmar un archivo (prueba de autenticidad)
gpg --sign --output informe.sig informe.txt

# Firma separada (más común en la práctica)
gpg --detach-sign --output informe.txt.sig informe.txt

# Verificar la firma
gpg --verify informe.txt.sig informe.txt

# Simular alteración y verificar de nuevo
echo 'Dato modificado' >> informe.txt
```

```
gpg --verify informe.txt.sig informe.txt
```

4.5 Preguntas de reflexión

1. ¿Qué información aparece cuando verificas correctamente una firma GPG? ¿Y cuando la verificación falla?

Verificación correcta: **Verificación fallida:**

2. ¿Cómo garantiza GPG tanto la confidencialidad como la autenticidad en un solo mensaje? Describe el proceso:

Tu respuesta:

Ejercicio 5 HMAC y autenticación de mensajes

15 min
Nivel: Avanzado

5.1 Contexto

HMAC (Hash-based Message Authentication Code) combina una función hash con una clave secreta compartida. Garantiza tanto la integridad como la autenticidad de un mensaje sin usar criptografía asimétrica, siendo mucho más rápido.

5.2 Generación de HMAC

```
cd ~/practica-cripto

# Generar HMAC-SHA256 con clave compartida
openssl dgst -sha256 -hmac 'clave_compartida_secreta' mensaje.txt

# Guardar el HMAC
openssl dgst -sha256 -hmac 'clave_compartida_secreta' mensaje.txt >
mensaje.hmac
cat mensaje.hmac

# Verificar: regenerar y comparar
openssl dgst -sha256 -hmac 'clave_compartida_secreta' mensaje.txt

# Probar con clave diferente (debería dar HMAC diferente)
openssl dgst -sha256 -hmac 'clave_diferente' mensaje.txt
```

5.3 Escenario práctico

Simula el siguiente escenario: el servidor envía un mensaje al cliente junto con su HMAC. El cliente verifica que el mensaje no fue alterado en tránsito.

```
# SERVIDOR: genera mensaje y HMAC
echo 'Transferencia autorizada: 500 EUR a cuenta ES12345' > transaccion.txt
CLAVE='secreto_compartido_TLS'
openssl dgst -sha256 -hmac "$CLAVE" transaccion.txt | awk '{print $2}' >
transaccion.hmac

echo 'Mensaje enviado:'
cat transaccion.txt
echo 'HMAC enviado:'
cat transaccion.hmac

# ATACANTE: intenta modificar el mensaje
sed -i 's/500 EUR/50000 EUR/' transaccion.txt

# CLIENTE: verifica el HMAC
HMAC_RECIBIDO=$(cat transaccion.hmac)
HMAC_CALCULADO=$(openssl dgst -sha256 -hmac "$CLAVE" transaccion.txt | awk
'{print $2}')

if [ "$HMAC_RECIBIDO" = "$HMAC_CALCULADO" ]; then
  echo 'VERIFICACION OK: mensaje integro'
else
  echo 'ALERTA: mensaje alterado!'
fi
```

5.4 Preguntas de reflexión

1. ¿Cuál es la diferencia entre un hash simple (SHA-256) y un HMAC? ¿Qué problema adicional resuelve el HMAC?

Tu respuesta:

2. ¿En qué protocolos de red has estudiado que se usa HMAC? Nombra al menos dos:

Tu respuesta: